Détection de cerceaux par CNN

Partie vision du projet de vol autonome d'un drone Tello

Rapport technique et pédagogique : le code écrit, les notions de CNN et de PyTorch qu'il met en œuvre, et les résultats du premier entraînement

29 septembre 2026

Code : ~/Documents/tello (paquet Python tellovision)

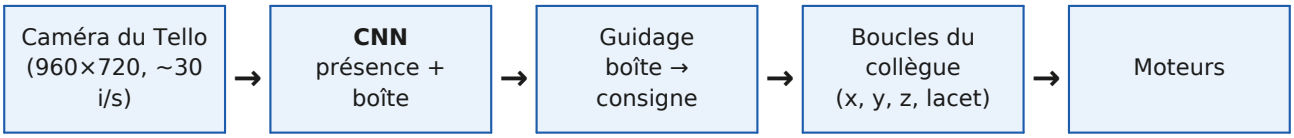
Sommaire

	Section
1	Objectif et place de la vision dans le projet
2	Organisation du code
3	Les données : annotation, découpage, Dataset et DataLoader
4	Augmentation, duplication et images de synthèse
5	Le réseau de neurones convolutif (CNN)
6	La fonction de perte
7	La boucle d'entraînement
8	Les métriques
9	Résultats du premier entraînement
10	Limites et prochaines étapes
A	Annexe : commandes et glossaire

1. Objectif et place de la vision dans le projet

Le projet consiste à faire traverser un circuit de cerceaux à un drone DJI Tello de façon autonome. Le travail est partagé en deux :

- **Asservissement (collègue)** : identification des modèles du drone et boucles fermées qui stabilisent la trajectoire sur x, y, z et le lacet.
- **Vision (ce rapport)** : regarder l'image de la caméra, trouver le cerceau, et en déduire la **consigne** à envoyer à ces boucles.



On a retenu l'option d'une **consigne recalculée à chaque image** (10 à 20 fois par seconde). Les dérives de l'odométrie du drone sont ainsi corrigées en permanence par la vision. Pour la traversée finale, quand le cerceau sort du champ de la caméra, une avance d'environ 1 m est commandée d'un bloc.

Le réseau ne renvoie pas seulement « il y a un cerceau » : il renvoie aussi **où** il se trouve, sous forme d'une boîte englobante. Le décalage horizontal de la boîte donne la correction de cap, le décalage vertical la correction d'altitude, et sa taille (le diamètre réel du cerceau étant connu) la distance.

> **Périmètre.** Ce rapport couvre la chaîne d'apprentissage : données → modèle → entraînement → évaluation. Les modules de vol (detector.py, guidance.py, mission.py) restent à écrire.

2. Organisation du code

La structure reprend la « base de code modulaire » du cours d'introduction à PyTorch de l'Inria (bibliothèque torchtmpl) : un paquet Python installable, un module par rôle, et un fichier de configuration YAML qui regroupe tous les réglages. On change une expérience en modifiant le YAML, pas le code.

```
~/Documents/tello/
├── pyproject.toml            # rend le paquet installable : pip install -e .
├── configs/detect.yaml       # TOUS les réglages (données, augmentations, modèle, optimiseur)
├── tellovision/              # le paquet (équivalent de torchtmpl)
│   ├── data.py               # lecture des annotations, découpage, augmentations, Dataset
│   ├── models.py             # architectures : HoopNet, SimpleCNN, build_model()
│   ├── losses.py             # fonction de perte HoopLoss
│   ├── metrics.py            # métriques (exactitude, IoU, erreurs de centre et de taille)
│   ├── utils.py              # train_one_epoch, evaluate, ModelCheckpoint
│   └── main.py               # point d'entrée : train / test
├── tools/                    # scripts interactifs ou liés au drone
│   ├── capture_live.py       # capture d'images depuis le flux du Tello
│   ├── annotate_hoops.py     # annotation des boîtes (avec marge grise)
│   ├── show_augmentations.py # galerie de contrôle des augmentations
│   └── Tello_CL_Z_photo.py   # capture à plusieurs hauteurs / caps
├── captures/                 # hoop/, no_hoop/, sprites/, annotations.csv
├── logs/                     # un dossier par entraînement (poids, courbes, galeries)
└── docs/                     # ce rapport
```

Fichier	Rôle	Équivalent torchtmpl
data.py	Transformer les images annotées en tenseurs prêts pour le réseau	data.py
models.py	Définir le réseau ; build_model(cfg) le construit d'après le YAML	models.py
losses.py	Mesurer l'erreur du réseau sur un lot d'images	(dans main.py)
metrics.py	Mesurer la qualité de façon interprétable pour le guidage	—
utils.py	Boucles d'entraînement et d'évaluation, sauvegarde du meilleur modèle	utils.py
main.py	Enchaîner le tout : python -m tellovision.main config.yaml train	main.py

3. Les données

3.1 Capture et annotation

Les images viennent de la caméra du Tello (960×720). Elles sont rangées dans captures/hoop/ (un cerceau visible) et captures/no_hoop/ (aucun cerceau). Au moment de ce rapport : **58 images de cerceau et 19 images vides**, prises lors de deux sessions dans le même couloir.

Pour chaque image de cerceau, on dessine une boîte avec tools/annotate_hoops.py. Le résultat est stocké dans captures/annotations.csv :

```
file,present,x1,y1,x2,y2,img_w,img_h
hoop/frame_0002_1790587386373.jpg,1,519.8,-46.1,741.6,172.8,960,720
no_hoop/frame_0024_1790587329647.jpg,0,0,0,0,0,960,720
```

Boîtes « amodales ». Dans 24 images sur 58, le cerceau était coupé par le bord de l'image. Plutôt que de supprimer ces images (ce que le drone verra justement en vol, quand le cerceau est au-dessus de lui ou très proche), on annote le **cerceau entier**, même hors champ. Les coordonnées peuvent alors être négatives (ci-dessus, y1 = −46). Pour le guidage c'est idéal :

- le centre de la boîte hors de l'image indique directement la direction à prendre (y < 0 : monter) ;
- la taille de la boîte reste celle du vrai cerceau, donc l'estimation de distance reste juste.

L'outil d'annotation affiche pour cela une marge grise autour de l'image, et une ellipse jaune inscrite dans la boîte que l'on cale sur l'arc visible. L'option --redo-edge rouvre uniquement les boîtes qui touchaient le bord.

3.2 Découpage entraînement / validation

On garde une partie des images de côté (la **validation**) pour mesurer si le réseau généralise à des images qu'il n'a jamais vues. Un piège important : les images capturées en rafale se ressemblent presque pixel pour pixel. Si l'on tire les images de validation au hasard, chacune a une quasi-jumelle dans l'entraînement, et le score mesuré est faussement bon (on parle de **fuite de données**).

split_by_blocks() regroupe donc les images prises à moins de 2 secondes d'intervalle (block_seconds) en **blocs**, et envoie des blocs entiers d'un côté ou de l'autre, séparément pour les deux classes. Résultat : 58 images d'entraînement (43 cerceaux, 15 vides) et 19 de validation (15 cerceaux, 4 vides), sans aucune image commune.

3.3 Dataset et DataLoader : les deux briques de PyTorch

PyTorch sépare deux rôles :

- **Dataset** : sait renvoyer un exemple. Il suffit d'écrire deux méthodes, __len__ (combien d'exemples) et __getitem__(k) (le k-ième exemple).
- **DataLoader** : assemble des exemples en **lots** (batches), les mélange à chaque époque (shuffle) et les prépare en parallèle dans plusieurs processus (num_workers).

```
class HoopDataset(Dataset):
    def __len__(self):
        return len(self.index)

    def __getitem__(self, k):
        img, box, present = self.load(k)          # lecture (+ augmentation si entraînement)
        h, w = img.shape[:2]
        target = encode_box(box, w, h) if present else np.zeros(4, np.float32)
        return (preprocess(img, self.input_size),  # image  -> tenseur (3, 192, 256)
                torch.tensor(float(present)),      # 1.0 ou 0.0
                torch.from_numpy(target))          # [cx, cy, w, h]
```

Du tableau d'image au tenseur (preprocess). OpenCV lit une image en BGR, sous forme d'un tableau (hauteur, largeur, 3) d'entiers de 0 à 255. Le réseau attend au contraire :

- une taille fixe, ici 256×192 (même rapport 4:3 que la caméra) ;
- l'ordre des couleurs RGB (celui des images d'entraînement d'ImageNet) ;
- des réels **normalisés** : (x/255 − moyenne) / écart-type, avec la moyenne et l'écart-type d'ImageNet, car le réseau pré-entraîné a appris sur des images normalisées ainsi ;
- l'ordre des dimensions (canaux, hauteur, largeur), dit **CHW**, convention de PyTorch.

Le DataLoader empile ensuite 16 exemples : un lot d'images est un tenseur de forme **(16, 3, 192, 256)**.

> **À retenir.** preprocess() sera réutilisée telle quelle en vol. Si l'image fournie au réseau en vol n'est pas préparée exactement comme à l'entraînement (couleurs, taille, normalisation), ses prédictions n'ont plus de sens.

3.4 Codage de la cible

La boîte (x1, y1, x2, y2) en pixels est convertie par encode_box() en **[cx, cy, w, h]** : centre, largeur et hauteur, divisés par la taille de l'image. Ces valeurs sont indépendantes de la

résolution et du même ordre de grandeur (autour de 0 à 1), ce qui facilite l'apprentissage. Un centre à cx = 0,5 signifie « au milieu de l'image » ; cy = −0,1 signifie « juste au-dessus du cadre ».

4. Augmentation, duplication et images de synthèse

Avec 58 images de cerceau, un réseau apprend vite **par cœur** : il retient le couloir, la porte, la personne qui tient le support, au lieu de la forme du cerceau. C'est le **surapprentissage**. L'augmentation de données consiste à modifier aléatoirement chaque image à chaque lecture, pour que le réseau ne voie jamais deux fois exactement la même.

4.1 Les transformations

Transformation	Paramètres (detect.yaml)	Ce qu'elle simule / pourquoi
Remplacement du fond	p_background 0,4	L'anneau est découpé et collé sur une image vide : le décor et le support disparaissent, seul l'anneau reste.
Miroir horizontal	p_flip 0,5	Cerceau vu depuis l'autre côté ; double la variété.
Zoom + décalage	scale 0,6–1,3 ; translate 20 %	Cerceau plus ou moins loin, à différents endroits de l'image, y compris partiellement hors champ.
Couleur / lumière	brightness, contrast, saturation, hue 4°	Contre-jour, exposition automatique de la caméra. La teinte bouge peu.
Flou de bougé	p 0,3 ; noyau ≤ 11 px	Le drone qui bouge ou vibre.
Bruit	p 0,3 ; écart-type ≤ 8	Bruit du capteur en faible lumière.
Rectangle masqué	p 0,3 ; ≤ 15 % de l'image	Occultation partielle (personne, objet).
Compression JPEG	p 0,5 ; qualité 35–80	Artefacts du flux vidéo transmis en Wi-Fi.

Les transformations géométriques déplacent aussi la boîte : après un miroir, par exemple, x1' = largeur − x2. Les transformations sont appliquées **uniquement à l'entraînement** ; la validation reste brute, pour mesurer les performances sur de vraies images.

4.2 La duplication, sans fichier supplémentaire

Plutôt que d'écrire des copies modifiées sur le disque, HoopDataset répète chaque image dans son index (repeats : 4 fois pour un cerceau, 8 fois pour une image vide, pour rééquilibrer les classes). Comme les augmentations sont tirées au hasard à chaque lecture, chaque répétition est différente, et c'est encore vrai d'une époque à l'autre : le nombre de variantes vues est illimité.

4.3 Images de synthèse : apprendre la forme

Trois photos de cerceaux sur fond blanc (captures/sprites/) servent de **modèles de forme**. load_sprites() retire le blanc pour ne garder que l'anneau, puis synthesize() le colle sur une vraie image vide du Tello, en tirant au hasard sa taille (12 à 60 % de la largeur), son aplatissement (cerceau vu de biais), un roulis de ±8°, sa position et sa couleur (y compris gris métallique). La boîte est calculée exactement à partir des pixels de l'anneau déformé. 120 images de ce type sont ajoutées à chaque époque.

Comme seule la forme est constante d'une image à l'autre (le fond, la couleur, la taille changent), le réseau est poussé à reconnaître la forme d'un anneau plutôt que l'environnement.

Images corrompues. Trois images vides présentaient de gros blocs pixellisés dus à des pertes de paquets Wi-Fi. La fonction blockiness() les détecte (rapport entre le gradient mesuré sur les

frontières de blocs de 16 px et ailleurs : environ 1,2 pour une image normale, plus de 2,5 si elle est corrompue). Elles restent des exemples négatifs, mais ne servent plus de fond.

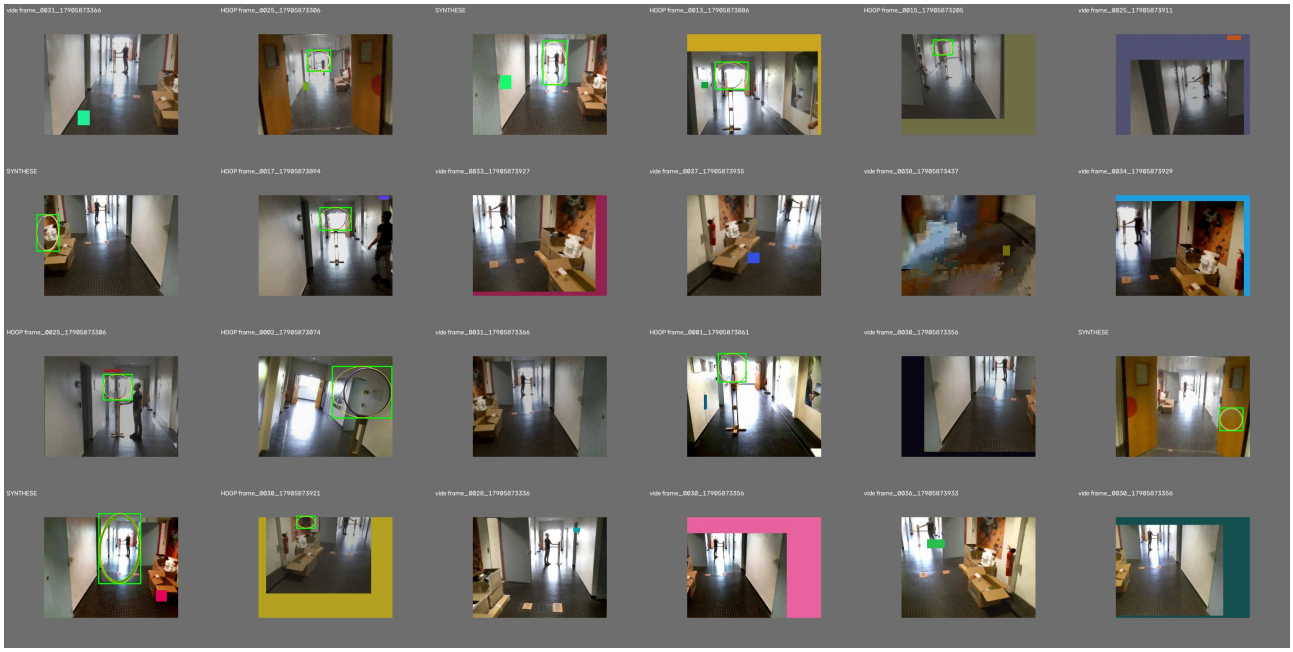


Figure 1 – Exemples vus par le réseau à l'entraînement (tools/show_augmentations.py). Vert : boîte ; jaune : ellipse inscrite. La marge grise montre les boîtes qui débordent de l'image.

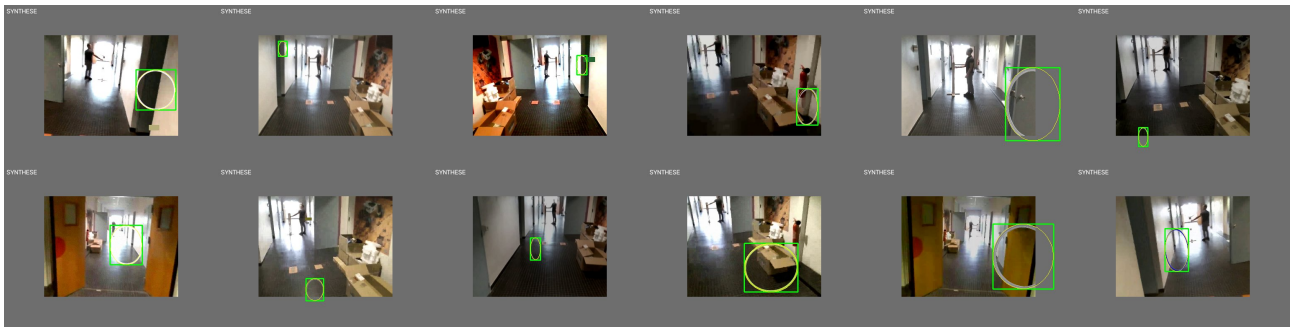


Figure 2 – Images de synthèse : anneau recoloré et déformé, collé sur de vraies images vides du Tello.

5. Le réseau de neurones convolutif (CNN)

5.1 Les briques d'un CNN

Brique	Ce qu'elle fait	Dans PyTorch
Convolution	Un petit filtre (par exemple 3×3) glisse sur l'image et calcule en chaque position une somme pondérée des pixels voisins. Les poids du filtre sont appris. Chaque filtre produit une **carte de caractéristiques** (feature map) qui s'allume là où son motif est présent (un bord, un coin, un arc...). Les mêmes poids servent partout dans l'image.	nn.Conv2d
Non-linéarité	ReLU(x) = max(0, x). Sans elle, empiler des couches reviendrait à une seule opération linéaire.	nn.ReLU
Normalisation par lot	Recentre et remet à l'échelle les activations de chaque canal ; stabilise et accélère l'apprentissage.	nn.BatchNorm2d
Sous-échantillonnage	Pooling ou convolution à pas 2 : divise la résolution par 2. Chaque neurone « voit » alors une plus grande zone de l'image (**champ réceptif**).	nn.MaxPool2d, stride=2

Brique	Ce qu'elle fait	Dans PyTorch
Couche dense	Chaque sortie est une combinaison de toutes les entrées ; sert à produire les nombres finaux.	nn.Linear
Dropout	Met à zéro une partie des neurones au hasard pendant l'entraînement : limite le surapprentissage.	nn.Dropout

En empilant ces briques, les premières couches détectent des motifs simples (bords, textures), les suivantes des combinaisons (arcs, contours courbes), les dernières des objets entiers. La résolution spatiale diminue pendant que le nombre de canaux augmente.

5.2 Le transfert d'apprentissage

Entraîner un CNN de zéro demande des milliers d'images. On part donc d'un réseau **pré-entraîné** sur ImageNet (1,2 million d'images, 1 000 classes) : **MobileNetV3-small**, conçu pour être léger et rapide (il tourne en temps réel sur le CPU d'un portable). Ses couches de convolution savent déjà extraire bords, formes et textures ; il reste à leur apprendre ce qu'est un cerceau.

On ne garde que sa partie convolutive (net.features, appelée **backbone**), et on remplace sa sortie de classification par notre propre **tête**. Le backbone apprend 10 fois plus lentement que la tête (backbone_lr_mult 0,1) pour ne pas détruire ce qu'il sait déjà.

5.3 Architecture de HoopNet

Étape	Opération	Forme du tenseur	Paramètres
Entrée	lot de 16 images	(16, 3, 192, 256)	—
Backbone	MobileNetV3-small, pas total 32	(16, 576, 6, 8)	0,93 M
Réduction	Conv 1×1 576→64 + BatchNorm + ReLU	(16, 64, 6, 8)	37 k
Aplatissement	Flatten	(16, 3072)	—
Dense	Dropout + Linear 3072→256 + ReLU	(16, 256)	0,79 M
Sortie	Dropout + Linear 256→5	(16, 5)	1,3 k

Le backbone transforme chaque image 256×192 en une grille de 8×6 cases, chacune décrite par 576 nombres. Chaque case résume une zone d'environ 32×32 pixels de l'image d'entrée.

Pourquoi ne pas faire de moyenne globale ? Les réseaux de classification moyennent d'habitude cette grille (global average pooling) pour obtenir un seul vecteur. Pour savoir s'il y a un cerceau c'est suffisant, mais la moyenne efface où il se trouve. Notre tête réduit donc le nombre de canaux (convolution 1×1, 576 → 64) puis **aplatit** la grille : la couche dense reçoit chaque case à sa position, et peut apprendre « cerceau en haut à gauche » différemment de « en bas à droite ».

Les 5 sorties sont contraintes à des plages utiles :

```
def forward(self, f):
    out = self.fc(self.reduce(f))                  # (B, 5) nombres réels quelconques
    logit  = out[:, :1]                            # présence : score brut (logit)
    center = 2 * torch.sigmoid(out[:, 1:3]) - 0.5  # cx, cy dans [-0.5, 1.5] (hors cadre possible)
    size   = 2 * torch.sigmoid(out[:, 3:5])        # w, h dans [0, 2] (cerceau très proche)
    return torch.cat([logit, center, size], dim=1)
```

La sigmoïde $\sigma(x) = 1/(1+e^{-x})$ ramène n'importe quel réel entre 0 et 1 ; on l'étire ensuite pour autoriser les boîtes amodales. La présence reste un **logit** (un score non borné) : la probabilité s'obtient par σ(logit), et c'est la fonction de perte qui applique la sigmoïde (voir §6).

5.4 SimpleCNN : une référence apprise de zéro

models.py contient aussi un petit CNN sans pré-entraînement : 5 blocs Conv 3×3 → BatchNorm → ReLU → MaxPool (16, 32, 64, 128, 128 canaux), suivis de la même tête. Il suffit de mettre class: SimpleCNN dans le YAML. Le comparer à HoopNet sur les mêmes données est la meilleure façon de mesurer l'apport du transfert d'apprentissage (expérience pas encore faite).

6. La fonction de perte

La perte (loss) est le nombre que l'entraînement cherche à faire diminuer. Elle combine les deux tâches du réseau :

perte totale = perte de présence + 5 × perte de boîte

6.1 Présence : entropie croisée binaire

Pour une image de cible y (1 = cerceau, 0 = vide) et une probabilité prédite p = σ(logit) :

$L_{cls} = -[\, y \cdot \log(p) + (1 - y) \cdot \log(1 - p) \,]$

Si le cerceau est présent (y = 1) et que le réseau répond p = 0,99, la perte vaut 0,01 ; s'il répond p = 0,01, elle vaut 4,6. Le réseau est fortement pénalisé quand il se trompe avec assurance. PyTorch fournit binary_cross_entropy_with_logits, qui prend directement le logit : la sigmoïde et le logarithme y sont calculés ensemble de façon numériquement stable. Le paramètre pos_weight permet de pénaliser davantage les cerceaux manqués.

6.2 Boîte : Smooth L1, seulement si un cerceau est présent

Une image vide n'a pas de boîte : on ne calcule la perte de boîte que sur les exemples positifs (masque present > 0.5). La **Smooth L1** se comporte comme un carré pour les petites erreurs (précision fine) et comme une valeur absolue pour les grandes (une annotation ratée ne domine pas tout l'apprentissage).

```
l_cls = F.binary_cross_entropy_with_logits(out[:, 0], present, pos_weight=self.pos_weight)
pos = present > 0.5
l_box = F.smooth_l1_loss(out[pos, 1:], target[pos], beta=0.05) if pos.any() else 0
return l_cls + self.box_weight * l_box, l_cls, l_box
```

Le poids 5 (box_weight) compense le fait que les erreurs de boîte (quelques centièmes) sont numériquement plus petites que la perte de présence.

7. La boucle d'entraînement

7.1 Les cinq lignes essentielles

Tout l'apprentissage tient dans cette boucle, répétée pour chaque lot (utils.train_one_epoch) :

```
for x, present, target in loader:
    out = model(x)                                  # 1. propagation avant : prédictions
    loss, l_cls, l_box = loss_fn(out, present, target)  # 2. erreur
    optimizer.zero_grad()                           # 3. efface les gradients du lot précédent
    loss.backward()                                 # 4. rétropropagation : dérivée de la perte
                                                    #    par rapport à chacun des 1,75 M poids
    optimizer.step()                                # 5. mise à jour des poids dans le sens qui
                                                    #    fait baisser la perte
```

PyTorch enregistre automatiquement toutes les opérations effectuées à l'étape 1 (**autograd**), ce qui lui permet de calculer tous les gradients en un seul appel à backward(). L'étape 3 est nécessaire car PyTorch additionne les gradients d'un appel à l'autre.

7.2 Mode entraînement et mode évaluation

- model.train() : le Dropout est actif, la BatchNorm met à jour ses statistiques.
- model.eval() : le Dropout est désactivé, la BatchNorm utilise les statistiques apprises. À utiliser pour la validation et en vol.
- @torch.no_grad() sur evaluate() : pas de mémorisation pour les gradients, plus rapide et moins gourmand en mémoire.

7.3 Optimiseur, calendrier, sauvegarde

Élément	Choix	Explication
Optimiseur	AdamW, lr 0,001 (tête), 0,0001 (backbone)	Adam adapte le pas de chaque poids selon l'historique de ses gradients ; le « W » ajoute une pénalité sur la taille des poids (weight decay 1e-4) qui limite le surapprentissage.
Calendrier du pas	Cosinus sur 40 époques	Le pas d'apprentissage diminue progressivement : grandes corrections au début, ajustements fins à la fin.
Sauvegarde	ModelCheckpoint	Enregistre les poids **et la configuration** à chaque fois que la perte de validation s'améliore : on garde le meilleur modèle, pas le dernier.
Arrêt anticipé	patience 10	Stoppe l'entraînement si la validation ne s'améliore plus pendant 10 époques.
Journal	TensorBoard	Courbes de toutes les pertes et métriques : tensorboard --logdir logs.

Chaque entraînement crée son propre dossier (logs/HoopNet_0, _1, ...) contenant la configuration utilisée, le meilleur modèle et ses courbes : les expériences restent comparables.

8. Les métriques

La perte sert à entraîner, mais elle est difficile à interpréter. metrics.py calcule des grandeurs parlantes pour le guidage (seuil de présence : $p \geq 0{,}5$) :

Métrique	Définition	Lien avec le vol
Exactitude	Part d'images bien classées (cerceau / vide)	Détecter un cerceau quand il y en a un.
Précision	Parmi les « cerceau » prédits, part de vrais cerceaux	Faible : le drone fonce vers des fantômes.
Rappel	Parmi les vrais cerceaux, part détectée	Faible : le drone perd le cerceau.
IoU	Aire de l'intersection / aire de l'union des deux boîtes (1 = parfait)	Qualité globale de la boîte.
Erreur du centre	Distance entre centres, en px d'une image 960×720	Erreur de cap et d'altitude.
Erreur de taille	$\lvert w \text{ prédit} - w \text{ vrai} \rvert / w \text{ vrai}$	Erreur relative sur la distance au cerceau.

9. Résultats du premier entraînement

Premier entraînement de HoopNet (logs/HoopNet_0) : 412 exemples par époque (292 réels augmentés + 120 de synthèse), environ 25 s par époque sur le CPU, arrêt anticipé à l'époque 36. Meilleur modèle : époque 26.

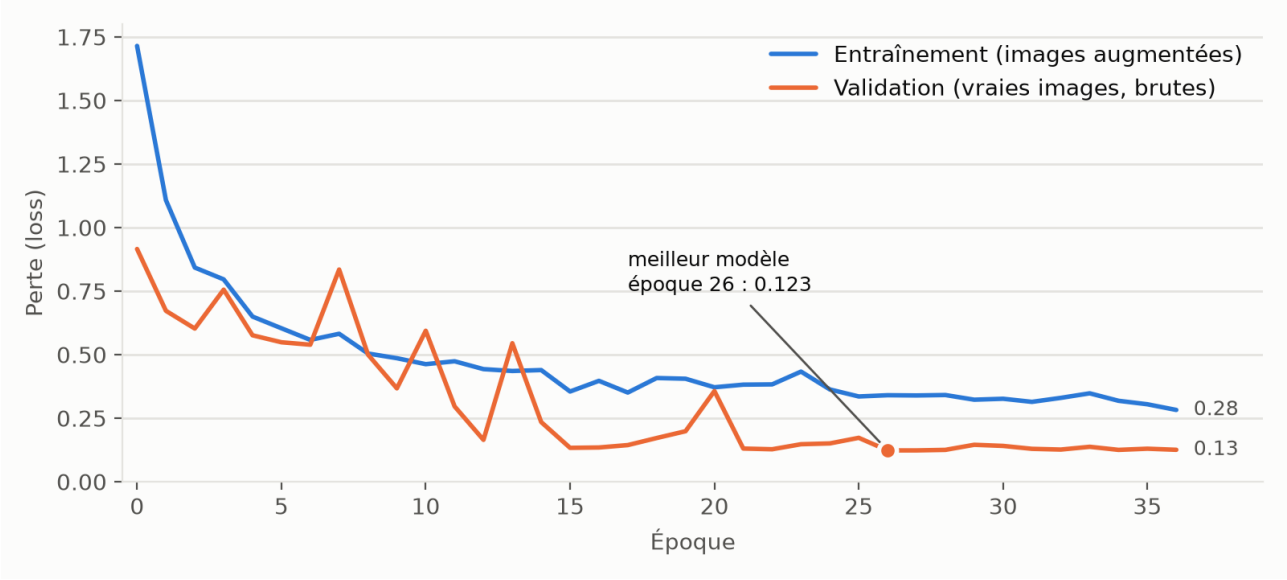


Figure 3 – Perte d'entraînement et de validation au fil des époques.

Pourquoi la perte de validation est-elle plus basse que celle d'entraînement ? Ce n'est pas une anomalie : l'entraînement se fait sur des images volontairement rendues difficiles (augmentations, synthèse) avec le Dropout actif, alors que la validation utilise des images brutes et le réseau complet. Les fortes oscillations de la validation viennent de sa petite taille : avec 19 images, une seule erreur change nettement le score.

Métrique (validation, 19 images)	Valeur	Lecture
Exactitude / précision / rappel	100 %	Tous les cerceaux et toutes les images vides reconnus.
IoU moyen	0,50	Boîte approximative.
Erreur du centre	55 px	≈ 6 % de la largeur, soit 4 à 5° de cap.
Erreur de taille	30 %	≈ 30 % d'erreur sur la distance : insuffisant pour la traversée.

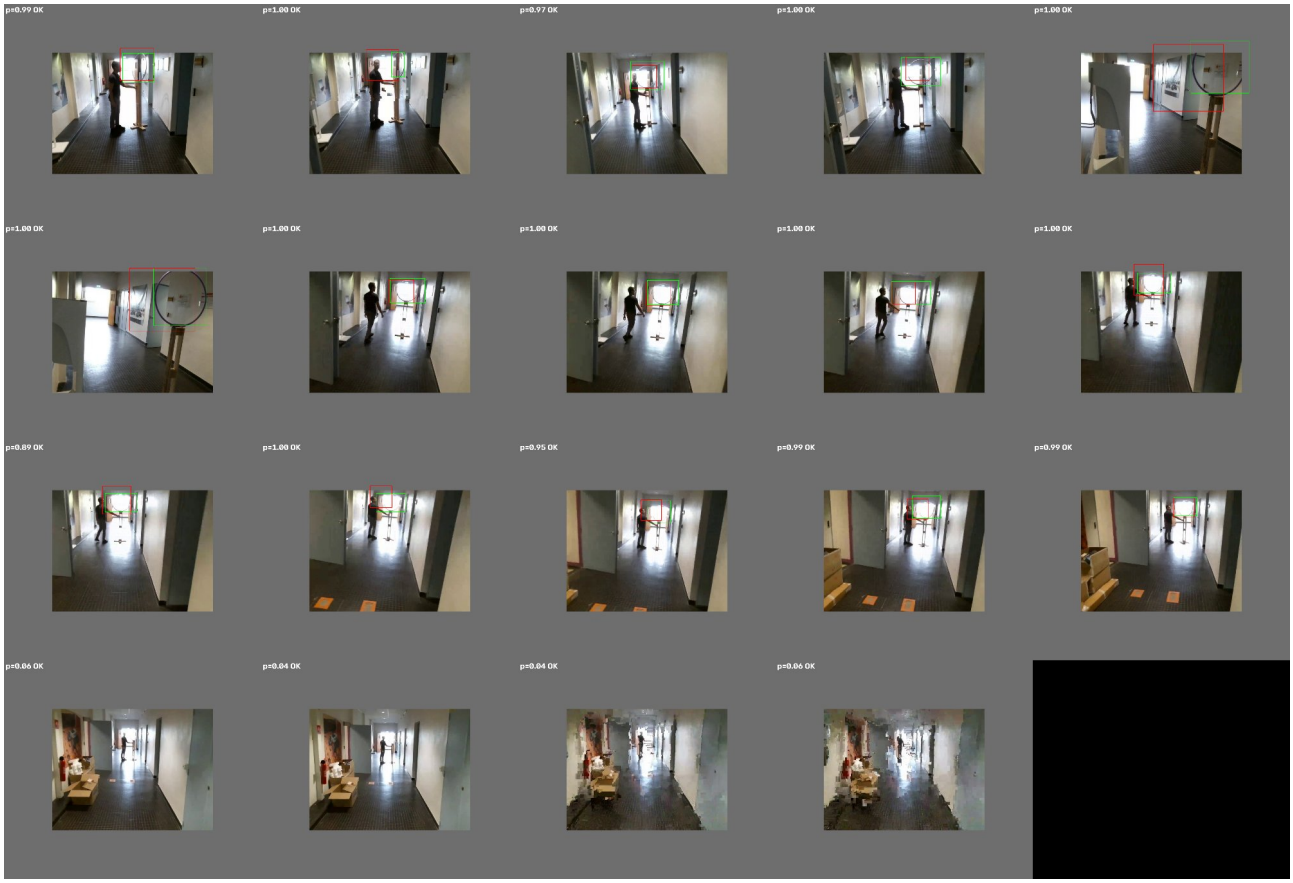


Figure 4 – Prédictions sur la validation (python -m tellovision.main logs/HoopNet_0 test). Vert : annotation ; rouge : prédiction ; p : probabilité de présence.

Analyse.

- **La détection de présence fonctionne**, y compris sur les deux images corrompues par le Wi-Fi (p ≈ 0,05).
- **Un biais est visible** : la boîte rouge est souvent décalée vers la gauche, sur la personne qui se tient à côté du support. Le réseau a appris en partie « cerceau = là où il y a quelqu'un », parce que cette personne apparaît sur presque toutes les images de cerceau. C'est exactement le type d'apprentissage de l'environnement que l'augmentation cherche à éviter ; elle l'a limité sans le supprimer.
- **La validation est trop petite et trop semblable à l'entraînement** (même couloir, même jour) pour garantir le comportement dans un autre lieu.

10. Limites et prochaines étapes

Priorité	Action	Effet attendu
1	Nouvelles captures : autres lieux, cerceau sans personne à côté, personnes sans cerceau, beaucoup d'images vides	Corrige le biais « personne », enrichit les fonds de synthèse, rend la validation fiable.
2	Entrée plus grande : input_size [320, 240]	L'anneau est un trait fin qui disparaît presque à 256×192.
3	Comparer HoopNet et SimpleCNN	Mesurer l'apport du pré-entraînement (bonne expérience pédagogique).
4	Tête en carte de chaleur (type CenterNet)	Localisation plus précise que la régression par couche dense ; changement plus lourd.

Priorité	Action	Effet attendu
5	detector.py + live_demo.py	Flux du Tello → CNN → boîte et consigne affichées en direct, moteurs coupés, avant tout vol.

Pour relancer un entraînement après de nouvelles captures, il suffit d'annoter les nouvelles images puis de relancer la commande d'entraînement : le découpage, les augmentations et la synthèse s'adaptent automatiquement aux nouvelles données.

Annexe A – Commandes

```
cd ~/Documents/tello && source venv/bin/activate

python tools/capture_live.py                        # capturer (trier ensuite dans hoop/ ou no_hoop/)
python tools/annotate_hoops.py                      # annoter les nouvelles images
python tools/annotate_hoops.py --redo-edge          # reprendre les boîtes coupées au bord
python tools/show_augmentations.py configs/detect.yaml          # galerie d'entraînement
python tools/show_augmentations.py configs/detect.yaml --split valid

python -m tellovision.main configs/detect.yaml train  # entraîner (+ test automatique)
python -m tellovision.main logs/HoopNet_0 test        # réévaluer un modèle
tensorboard --logdir logs                             # courbes dans le navigateur
```

Annexe B – Glossaire

Terme	Définition
Tenseur	Tableau multidimensionnel de nombres (généralisation d'une matrice), manipulé par PyTorch.
Lot (batch)	Groupe d'exemples traités ensemble ; les poids sont mis à jour une fois par lot.
Époque	Un passage complet sur les données d'entraînement.
Logit	Score brut non borné ; la probabilité correspondante est σ(logit).
Gradient	Dérivée de la perte par rapport à un poids : indique dans quel sens le modifier.
Rétropropagation	Calcul de tous les gradients, de la sortie vers l'entrée du réseau.
Pas d'apprentissage (lr)	Taille des corrections appliquées aux poids à chaque lot.
Backbone	Partie convolutive du réseau, qui extrait les caractéristiques de l'image.
Tête (head)	Partie finale, qui transforme les caractéristiques en prédictions.
Transfert d'apprentissage	Réutiliser un réseau entraîné sur une autre tâche (ImageNet) comme point de départ.
Surapprentissage	Le réseau réussit sur l'entraînement mais pas sur des images nouvelles : il a mémorisé au lieu de généraliser.
Fuite de données	Présence en validation d'images quasi identiques à celles de l'entraînement, qui gonfle artificiellement les scores.
Augmentation	Transformations aléatoires des images d'entraînement pour en multiplier la variété.
Boîte amodale	Boîte du cerceau entier, y compris la partie hors de l'image.
IoU	Intersection sur union de deux boîtes, entre 0 (disjointes) et 1 (identiques).